

Running a PHP Application using Docker Compose

To Begin with the Lab

Summary of the Lab

In these Docker Compose labs, we learned how to containerize and manage applications using Docker Compose. We deployed a PHP application, integrated it with a MySQL database, and configured multi-container applications using a single docker-compose.yaml file. We explored Docker volumes to provide persistent storage for MySQL data, ensuring that data remained intact after restarting containers. We also configured custom Docker networks to enable communication between containers and used the depends_on directive to manage service startup order. Finally, we automated image creation using Docker Compose with the build option, allowing images to be built and deployed directly from the Dockerfile, simplifying the application deployment process and container management.

- Create a new directory
- Create a Dockerfile, index.php, docker-compose.yaml.
- Dockerfile:

```
FROM php:8.2-apache
WORKDIR /var/www/html
COPY . .
EXPOSE 80
```

- index.php

```
<?php
echo "<h1>Hello from Cloudfreeks!</h1>";

echo "<p><strong>Current Time:</strong> " . date("Y-m-d H:i:s") . "</p>";

echo "<p><strong>PHP Version:</strong> " . phpversion() . "</p>";

echo "<hr>";

echo "<h3>Contents of sample.txt</h3>";

if (file_exists("sample.txt")) {
```

```
    echo "<pre>";
    echo htmlspecialchars(file_get_contents("sample.txt"));
    echo "</pre>";
} else {
    echo "<p style='color:red;'>sample.txt not found!</p>";
}
?>
```

- docker-compose.yaml

```
version: "3.8"

services:

  webapp:

    image: phpapp:v1

    container_name: php-webapp

    ports:

      - "8080:80"
```

```
ubuntu@ip-172-31-45-84:~/php$ ls
Dockerfile  docker-compose.yaml  index.php  sample.txt
ubuntu@ip-172-31-45-84:~/php$
```

- Build the Docker Image

```
docker build -t phpapp:v1 .
```

```
ubuntu@ip-172-31-45-84:~/php$ sudo docker images

IMAGE          ID                DISK USAGE  CONTENT SIZE  EXTRA
phpapp:v1      b264ae522fea      707MB       176MB
ubuntu@ip-172-31-45-84:~/php$
```

- If any old container exists, remove it.
- Start Docker Compose

```
docker compose up -d
```

- If you get some error related to docker-compose, check the docker compose version and if not installed, install it.

```
sudo apt update
```

```
sudo apt install docker-compose-v2
```

- Now run the command again
- Verify Container

```
ubuntu@ip-172-31-45-84:~/php$ sudo docker compose up -d
WARN[0000] /home/ubuntu/php/docker-compose.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 2/2
 ✓ Network php_default Created
 ✓ Container php-webapp Started
ubuntu@ip-172-31-45-84:~/php$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS                               NAMES
e3c4e069999c   phpapp:v1      "docker-php-entrypoi..." About a minute ago Up About a minute 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   php-webapp
ubuntu@ip-172-31-45-84:~/php$
```

- Test the Application
- `http://<Public-IP>:8080`

Hello from Cloudfreeks!

Current Time: 2026-07-14 04:09:14

PHP Version: 8.2.32

Contents of sample.txt

Hello, Cloudfreeks!

- Now run the command

```
Sudo docker compose down
```

Create a MySQL Container Using Docker Compose

- Create a new directory
- And create index.php, Dockerfile, docker-compose.yaml and init.sql files
- index.php

```
<?php
error_reporting(E_ALL);
ini_set('display_errors', 1);

$host = "mysql-instance";
$username = "root";
$password = "1234";
$databse = "company_db";

$conn = new mysqli($host, $username, $password, $databse);
```

```
if ($conn->connect_error) {  
    die("Connection failed: " . $conn->connect_error);  
}  
  
$result = $conn->query("SELECT * FROM employees");  
?>
```

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>Employee Details</title>  
    <style>  
        body{  
            font-family: Arial, sans-serif;  
            margin:40px;  
            background:#f4f4f4;  
        }  
  
        h2{  
            text-align:center;  
        }  
  
        table{  
            width:70%;  
            margin:auto;  
            border-collapse:collapse;  
            background:white;  
        }
```

```
th{
    background:#007BFF;
    color:white;
    padding:10px;
}
```

```
td{
    padding:10px;
    text-align:center;
}
```

```
table,th,td{
    border:1px solid #ccc;
}
```

```
tr:nth-child(even){
    background:#f2f2f2;
}
```

```
.success{
    text-align:center;
    color:green;
    margin-bottom:20px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Employee Details</h2>
```

```
<div class="success">
```

```
Connected successfully to MySQL Database!
```

```
</div>
```

```
<table>
```

```
<tr>
```

```
    <th>Employee ID</th>
```

```
    <th>Employee Name</th>
```

```
    <th>Department</th>
```

```
    <th>Salary</th>
```

```
</tr>
```

```
<?php
```

```
while($row = $result->fetch_assoc()){
```

```
    echo "<tr>";
```

```
    echo "<td>{".$row['employee_id']."}</td>";
```

```
    echo "<td>{".$row['employee_name']."}</td>";
```

```
    echo "<td>{".$row['department']."}</td>";
```

```
    echo "<td>₹".number_format($row['salary'],2)."</td>";
```

```
    echo "</tr>";
```

```
}
```

```
$conn->close();
```

```
?>
```

```
</table>
```

```
</body>
```

```
</html>
```

- Dockerfile

```
FROM php:8.2-apache
```

```
COPY . /var/www/html/
```

```
RUN docker-php-ext-install mysqli
```

- docker-compose.yaml

```
services:
```

```
  webapp:
```

```
    build: .
```

```
    image: phpapp:v1
```

```
    container_name: php-webapp
```

```
  ports:
```

```
    - "8080:80"
```

```
  depends_on:
```

```
    - mysql
```

```
  networks:
```

```
    - app-network
```

```
  mysql:
```

```
    image: mysql:8.0
```

```
    container_name: mysql-instance
```

ports:

- "3306:3306"

environment:

MYSQL_ROOT_PASSWORD: "1234"

MYSQL_DATABASE: "company_db"

volumes:

- ./init.sql:/docker-entrypoint-initdb.d/init.sql

networks:

- app-network

networks:

app-network:

- create a new file init.sql

```
CREATE DATABASE IF NOT EXISTS company_db;
```

```
USE company_db;
```

```
CREATE TABLE employees (  
  employee_id INT PRIMARY KEY,  
  employee_name VARCHAR(100),  
  department VARCHAR(100),  
  salary DECIMAL(10,2)  
);
```

```
INSERT INTO employees VALUES  
(1,'Alice','HR',50000),
```



```
(2,'Bob','IT',65000),  
(3,'Charlie','Finance',70000);
```

- Delete the old images and container.
- Build the PHP Image

```
sudo docker compose up --build -d
```

```
=> => exporting manifest list sha256:6ac2efd3d141f67c3ea70c7ecc0d6742a6669e12e7305d601168  
=> => naming to docker.io/library/phpapp:v1  
=> => unpacking to docker.io/library/phpapp:v1  
=> [webapp] resolving provenance for metadata file  
[+] Running 4/4  
✓ webapp Built  
✓ Network app1_app-network Created  
✓ Container mysql-instance Started  
✓ Container php-webapp Started  
ubuntu@ip-172-31-45-84:~/app1$
```

- now go to the browser and check it

Employee Details			
Connected successfully to MySQL Database!			
Employee ID	Employee Name	Department	Salary
1	Alice	HR	₹50,000.00
2	Bob	IT	₹65,000.00
3	Charlie	Finance	₹70,000.00

- Stop the services.

```
sudo docker compose down
```

Docker Compose Volumes

- List volumes.

```
sudo docker volume ls
```

- Delete the existing volume.

```
sudo docker volume rm mysql-data
```

- Remove dangling volumes.

```
sudo docker volume prune
```

- Create the volume.

```
sudo docker volume create mysql-data
```

```

ubuntu@ip-172-31-45-84:~/app2$ sudo docker volume ls
DRIVER      VOLUME NAME
local       47a6b973ebb41c5db53bf05544a9e857f504f32da5f2394f65aa00c7d1f84645
local       app1_mysql-data
local       mysql-data
ubuntu@ip-172-31-45-84:~/app2$ sudo docker volume rm mysql-data
mysql-data
ubuntu@ip-172-31-45-84:~/app2$ sudo docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Volumes:
47a6b973ebb41c5db53bf05544a9e857f504f32da5f2394f65aa00c7d1f84645

Total reclaimed space: 197MB
ubuntu@ip-172-31-45-84:~/app2$ sudo docker volume create mysql-data
mysql-data
ubuntu@ip-172-31-45-84:~/app2$ █

```

- Modify docker-compose.yml
- Replace the existing file with the following:

services:

webapp:

build: .

image: phpapp:v1

container_name: php-webapp

ports:

- "8080:80"

depends_on:

- mysql

restart: always

networks:

- app-network

mysql:

image: mysql:8.0

container_name: mysql-instance

restart: always

ports:

- "3306:3306"

environment:

MYSQL_ROOT_PASSWORD: "1234"

MYSQL_DATABASE: "company_db"

volumes:

- mysql-data:/var/lib/mysql

- ./init.sql:/docker-entrypoint-initdb.d/init.sql

networks:

- app-network

volumes:

mysql-data:

networks:

app-network:

driver: bridge

- Docker Compose
 - Network app2_app-network Created
 - Volume app2_mysql-data Created
 - Container mysql-instance Started
 - Container php-webapp Started
- Running Containers
 - php-webapp Up
 - mysql-instance Up
- Docker Volume
 - local app2_mysql-data

```

ubuntu@ip-172-31-45-84:~/app2$ sudo docker compose up -d
[*] Running 4/4
✓ Network app2_app-network Created
0.1s
✓ Volume app2_mysql-data Created
0.0s
✓ Container mysql-instance Started
0.5s
✓ Container php-webapp Started
0.6s
ubuntu@ip-172-31-45-84:~/app2$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
0d684403b719   phpapp:v1 "docker-php-entrypoi..." 41 seconds ago Up 40 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   php-web
bd098d3ad1e3   mysql:8.0 "docker-entrypoint.s..." 41 seconds ago Up 41 seconds 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp   mysql-i
instance
ubuntu@ip-172-31-45-84:~/app2$ sudo docker volume ls
DRIVER    VOLUME NAME
local     appl_mysql-data
local     app2_mysql-data
local     mysql-data
ubuntu@ip-172-31-45-84:~/app2$

```

- Connect to MySQL

```
sudo docker exec -it mysql-instance mysql -uroot -p1234
```

- Select the Database

```
USE company_db;
```

- Verify Existing Data

```
SELECT * FROM employees;
```

- Expected:

```

Database changed
mysql> SELECT * FROM employees;
+-----+-----+-----+-----+
| employee_id | employee_name | department | salary |
+-----+-----+-----+-----+
|          1 | Alice         | HR         | 50000.00 |
|          2 | Bob           | IT         | 65000.00 |
|          3 | Charlie       | Finance    | 70000.00 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

- Insert One Record

```
INSERT INTO employees
```

```
VALUES
```

```
(4,'David','Marketing',55000);
```

- Verify

```
SELECT * FROM employees;
```

- You should now see 4 rows.

```
mysql> INSERT INTO employees
-> VALUES
-> (4, 'David', 'Marketing', 55000);
Query OK, 1 row affected (0.09 sec)

mysql> SELECT * FROM employees;
+-----+-----+-----+-----+
| employee_id | employee_name | department | salary |
+-----+-----+-----+-----+
| 1 | Alice | HR | 50000.00 |
| 2 | Bob | IT | 65000.00 |
| 3 | Charlie | Finance | 70000.00 |
| 4 | David | Marketing | 55000.00 |
+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

- Exit MySQL

```
exit
```

- Stop the Containers

```
sudo docker compose down
```

- Start Again

```
sudo docker compose up -d
```

- Go to the browser and search through IP address on port 8080
- You will see 4 rows instead of 3.

Employee Details			
Connected successfully to MySQL Database!			
Employee ID	Employee Name	Department	Salary
1	Alice	HR	₹50,000.00
2	Bob	IT	₹65,000.00
3	Charlie	Finance	₹70,000.00
4	David	Marketing	₹55,000.00

- Stop the Running Containers

```
sudo docker compose down
```

Working with Docker Compose Networks

- Remove the Existing Network (Optional)
- If you have previously created a network manually, remove it.

```
sudo docker network rm app-network
```

- If the network does not exist, Docker will display an error. You can ignore it.
- Verify Existing Docker Networks

```
sudo docker network ls
```

- Note: If a manually created network exists from a previous lab, it can be removed using:

```
sudo docker network rm <network-name>
```

- If no such network exists, proceed to the next step.

```
ubuntu@ip-172-31-45-84:~/app2$ sudo docker network ls
NETWORK ID      NAME                DRIVER  SCOPE
47a761bae113    appl_app-network    bridge  local
9d4418c11c65    bridge              bridge  local
c65eca012e95    host                host    local
6ef111b3abf5    none                null    local
ubuntu@ip-172-31-45-84:~/app2$ sudo docker network rm appl_app-network
appl_app-network
ubuntu@ip-172-31-45-84:~/app2$ sudo docker network ls
NETWORK ID      NAME                DRIVER  SCOPE
9d4418c11c65    bridge              bridge  local
c65eca012e95    host                host    local
6ef111b3abf5    none                null    local
ubuntu@ip-172-31-45-84:~/app2$
```

- Update docker-compose.yaml

```
services:
```

```
  webapp:
```

```
    build: .
```

```
    image: phpapp:v1
```

```
    container_name: php-webapp
```

```
  ports:
```

```
    - "8080:80"
```

```
  depends_on:
```

```
    - mysql
```

```
  restart: always
```

```
  networks:
```

```
    - app-network
```

```
mysql:
```

image: mysql:8.0

container_name: mysql-instance

restart: always

ports:

- "3306:3306"

environment:

MYSQL_ROOT_PASSWORD: "1234"

MYSQL_DATABASE: "company_db"

volumes:

- mysql-data:/var/lib/mysql
- ./init.sql:/docker-entrypoint-initdb.d/init.sql

networks:

- app-network

volumes:

mysql-data:

networks:

app-network:

driver: bridge

ipam:

config:

- subnet: 172.20.0.0/16

- Start the Containers

```
sudo docker compose up -d
```

- Verify the Running Containers

```
sudo docker ps
```

- Both containers should be in the Up state.

```
network app2_app-network
0.1s p-webapp Starting
[*] Running 3/3sql-instance Started
✓ Network app2_app-network Created
0.1s p-webapp Starting
✓ Container mysql-instance Started
0.5s
✓ Container php-webapp Started
0.4s
ubuntu@ip-172-31-45-84:~/app2$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS                               NAMES
d4edfa6c62a8   phpapp:v1 "docker-php-entrypoi..." 8 seconds ago Up 7 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   php-webapp
0a54cedcce7d   mysql:8.0 "docker-entrypoint.s..." 8 seconds ago Up 7 seconds 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp   mysql-instance
ubuntu@ip-172-31-45-84:~/app2$
```

- Verify the Docker Network

```
sudo docker network ls
```

- You should see a network similar to:
- app2_app-network
- Since your project directory is app2, Docker Compose prefixes the network name with the project name.

```
ubuntu@ip-172-31-45-84:~/app2$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS                               NAMES
d4edfa6c62a8   phpapp:v1 "docker-php-entrypoi..." 42 seconds ago Up 41 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   php-webapp
0a54cedcce7d   mysql:8.0 "docker-entrypoint.s..." 42 seconds ago Up 41 seconds 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp   mysql-instance
ubuntu@ip-172-31-45-84:~/app2$ sudo docker network ls
NETWORK ID     NAME      DRIVER  SCOPE
f8f825696c67   app2_app-network   bridge  local
9d4418c11c65   bridge      bridge  local
c65eca012e95   host        host    local
6ef111b3abf5   none        null    local
ubuntu@ip-172-31-45-84:~/app2$
```

- Inspect the Network

```
sudo docker network inspect app2_app-network
```

- Verify that both containers are connected to the network.
- You should see entries for:
- php-webapp
- mysql-instance


```

"Options": {},
"Labels": {
  "com.docker.compose.config-hash": "c82c167651634a1e6f0de7c2ed4eaa7a3a141d835f66b55777fa55cd8bf73ccf",
  "com.docker.compose.network": "app-network",
  "com.docker.compose.project": "app2",
  "com.docker.compose.version": "2.40.3"
},
"Containers": {
  "0a54cedcce7dalb98ac5973729b2be8a9260845da67045d273e16716c2516638": {
    "Name": "mysql-instance",
    "EndpointID": "cc300abfad5c91ea7f13e67715b9f807899d36944db31cdd8c4c1c8d61716786",
    "MacAddress": "1a:99:3d:16:9c:19",
    "IPv4Address": "172.20.0.2/16",
    "IPv6Address": ""
  },
  "d4edfa6c62a8841548d0f2f8121040b478db55f8dcb55fc519ee634d3ed44f8": {
    "Name": "php-webapp",
    "EndpointID": "16a71b050342454a25c0f36a5ead47f5eca6baabfe5a976318e620f038920c4c",
    "MacAddress": "42:1e:23:8c:dc:0f",
    "IPv4Address": "172.20.0.3/16",
    "IPv6Address": ""
  }
},
"Status": {
  "IPAM": {
    "Subnets": {
      "172.20.0.0/16": {
        "IPsInUse": 5,
        "DynamicIPsAvailable": 65531
      }
    }
  }
}

```

- Inspect the MySQL Container

```
sudo docker inspect mysql-instance
```

- Verify that the container has been assigned an IP address from the configured subnet:
- 172.20.0.x

```

"Links": null,
"Aliases": [
  "mysql-instance",
  "mysql"
],
"DriverOpts": null,
"GwPriority": 0,
"NetworkID": "f8f825696c67e830b1a20415fa704dd3fc0cc7ef6c533270426baba874128c0d",
"EndpointID": "cc300abfad5c91ea7f13e67715b9f807899d36944db31cdd8c4c1c8d61716786",
"Gateway": "172.20.0.1",
"IPAddress": "172.20.0.2",
"MacAddress": "1a:99:3d:16:9c:19",
"IPPrefixLen": 16,
"IPv6Gateway": "",
"GlobalIPv6Address": "",
"GlobalIPv6PrefixLen": 0,
"DNSNames": [
  "mysql-instance",
  "mysql",
  "0a54cedcce7d"
]
}
},
"ImageManifestDescriptor": {
  "mediaType": "application/vnd.oci.image.manifest.v1+json",
  "digest": "sha256:62fb722c78b24245ddff1796a0fcee4a49cc5b87e0aaaf20c92d1da9e0a2497b",
  "size": 3045,
  "annotations": {
    "com.docker.official-images.bashbrew.arch": "amd64",

```

- Verify the Application
- Open the browser:
- <http://<Public-IP>:8080>

Employee Details			
Connected successfully to MySQL Database!			
Employee ID	Employee Name	Department	Salary
1	Alice	HR	₹50,000.00
2	Bob	IT	₹65,000.00
3	Charlie	Finance	₹70,000.00
4	David	Marketing	₹55,000.00

- Stop the running containers.

```
sudo docker compose down
```

Running Multiple Containers Using Docker Compose

- Continue in the same directory but delete all the images and container.
- Modify the docker-compose.yaml File
- Create two services:
- db (MySQL)
- webapp (PHP)
- Configure both services to use the same Docker network.

services:

db:

image: mysql:8.0

container_name: mysql-instance

ports:

- "3306:3306"

environment:

MYSQL_ROOT_PASSWORD: "1234"

MYSQL_DATABASE: "company_db"

volumes:

- ./init.sql:/docker-entrypoint-initdb.d/init.sql

networks:

- app-network

webapp:

build: .

image: phpapp:v2

container_name: php-webapp

ports:

- "8080:80"

depends_on:

- db

networks:

- app-network

networks:

app-network:

driver: bridge

- Build the PHP Image (If Not Already Built)

```
sudo docker build -t phpapp:v2 .
```

- Verify the image.

```
sudo docker images
```

- Start the Application

```
sudo docker compose up -d
```

- Verify the Running Containers

```
sudo docker ps
```

```
ubuntu@ip-172-31-45-84:~/app2$ sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
php:8.2-apache	409948e94ce2	714MB	183MB	
phpapp:v2	64ef8bf69dab	708MB	176MB	

```
ubuntu@ip-172-31-45-84:~/app2$ sudo docker compose up -d
```

```
[+] Running 14/14
```

- ✓ db Pulled 12.7s
- ✓ 96d30d9fbee8 Pull complete 0.2s
- ✓ 4c8a3e0d4e4b Pull complete 0.2s
- ✓ a63160a5eda1 Pull complete 5.3s
- ✓ 7534d1db9f8d Pull complete 0.3s
- ✓ 49ec2dab01d9 Pull complete 12.3s
- ✓ ab24264a27e9 Pull complete 0.3s
- ✓ 297d04cfe470 Pull complete 0.2s
- ✓ edf85873f64e Pull complete 3.6s
- ✓ 6ef6c7b50a93 Pull complete 0.2s
- ✓ e3e5d1ac74c1 Pull complete 0.2s
- ✓ 0d74d296605b Pull complete 3.9s
- ✓ 796812c73292 Download complete 0.0s
- ✓ af166387641d Download complete 0.5s

```
[+] Running 3/3
```

- ✓ Network app2_app-network Created 0.1s
- ✓ Container mysql-instance Started 0.5s
- ✓ Container php-webapp Started 0.5s

```
ubuntu@ip-172-31-45-84:~/app2$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
559384fb7c92	phpapp:v2	"docker-php-entrypoi..."	40 seconds ago	Up 39 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	php-web
app						
10abe34d82c3	mysql:8.0	"docker-entrypoint.s..."	40 seconds ago	Up 39 seconds	0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp	mysql-i
instance						

- Navigate to browser and check, you will see the similar table
- Stop the Containers

```
sudo docker compose down
```

- Remove the Existing Docker Volumes
- List the existing Docker volumes.

```
sudo docker volume ls
```

- Remove the MySQL volume.

```
sudo docker volume rm mysql-data
```

- Remove the application volume.

```
sudo docker volume rm app2_mysql-data
```

```
ubuntu@ip-172-31-45-84:~/app2$ sudo docker volume ls
```

DRIVER	VOLUME NAME
local	55c2f5cdb36873a9f360bcf97f8calbcea4f3d130b881e2592ca6416ade056d5
local	app2_mysql-data
local	mysql-data

```
ubuntu@ip-172-31-45-84:~/app2$ sudo docker volume rm app2_mysql-data
```

```
app2_mysql-data
```

```
ubuntu@ip-172-31-45-84:~/app2$ sudo docker volume rm mysql-data
```

```
mysql-data
```

```
ubuntu@ip-172-31-45-84:~/app2$
```

- Modify docker-compose.yml
- Update the Docker Compose file to include two named volumes.

```
services:
```

webapp:

image: dotnetapp:v3

container_name: dotnet-instance

ports:

- "8080:80"

depends_on:

- db

networks:

- app-network

volumes:

- dotnet-data:/app/wwwroot/images

db:

image: mysqlapp:8.0

container_name: mysql-instance

ports:

- "3306:3306"

environment:

MYSQL_ROOT_PASSWORD: "1234"

MYSQL_DATABASE: "company_db"

volumes:

- mysql-data:/var/lib/mysql

networks:

- app-network

volumes:

mysql-data:

dotnet-data:

networks:

app-network:

driver: bridge

- Start the containers:

```
sudo docker compose up --build -d
```

- Verify:

```
sudo docker ps
```

- Verify the volumes:

```
sudo docker volume ls
```

```
ubuntu@ip-172-31-45-84:~/app2$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
929db7d0e156   phpapp:v2     "docker-php-entrypoi..." 53 seconds ago Up 52 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   php-web
app
85ef662806ca   mysql:8.0     "docker-entrypoint.s..." 53 seconds ago Up 52 seconds 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp   mysql-i
nstance
ubuntu@ip-172-31-45-84:~/app2$ sudo docker volume ls
DRIVER          VOLUME NAME
local          55c2f5cdb36873a9f360bcf97f8calbcea4f3d130b881e2592ca6416ade056d5
local          app2_mysql-data
local          app2_php-data
ubuntu@ip-172-31-45-84:~/app2$
```

- Stop the Running Containers

```
sudo docker compose down
```

- Open the Docker Compose File

```
nano docker-compose.yaml
```

- Modify docker-compose.yaml
- Add the depends_on clause under the webapp service.

services:

webapp:

build: .

image: phpapp:v2

container_name: php-webapp

ports:

- "8080:80"

depends_on:

- db

volumes:

- php-data:/var/www/html

networks:

- app-network

db:

image: mysql:8.0

container_name: mysql-instance

restart: always

ports:

- "3306:3306"

environment:

MYSQL_ROOT_PASSWORD: "1234"

```
MYSQL_DATABASE: "company_db"
```

```
volumes:
```

- mysql-data:/var/lib/mysql
- ./init.sql:/docker-entrypoint-initdb.d/init.sql

```
networks:
```

- app-network

```
volumes:
```

```
mysql-data:
```

```
php-data:
```

```
networks:
```

```
app-network:
```

```
driver: bridge
```

- Start the Containers

```
sudo docker compose up -d
```

- Verify the Running Containers

```
sudo docker ps
```

Building Docker Images Using Docker Compose

Summary of the Lab

In this lab, Docker Compose is configured to build the application image automatically using the Dockerfile. Instead of building the image manually with docker build, Docker Compose builds the image before creating the container.

- Verify the Existing Images

```
sudo docker images
```

- You should see your PHP image.
 - Example:
 - phpapp:v2

- php:8.2-apache
- Remove the Existing PHP Image

```
sudo docker rmi phpapp:v2
```

- Verify that the image has been removed.

```
sudo docker images
```

```
ubuntu@ip-172-31-45-84:~/app2$ sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
mysql:8.0	7dcddc01f13b	1.1GB	249MB	
php:8.2-apache	408948e94ce2	714MB	183MB	
phpapp:v2	465b675b734f	708MB	176MB	

```
ubuntu@ip-172-31-45-84:~/app2$ sudo docker rmi phpapp:v2
Untagged: phpapp:v2
Deleted: sha256:465b675b734fa6ffb4cf5cc6a33628923a0a8e819e13b0236507ff1617721df2
ubuntu@ip-172-31-45-84:~/app2$ sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
mysql:8.0	7dcddc01f13b	1.1GB	249MB	
php:8.2-apache	408948e94ce2	714MB	183MB	

```
ubuntu@ip-172-31-45-84:~/app2$
```

- Open docker-compose.yaml

```
nano docker-compose.yaml
```

- Verify the build Option
- The webapp service should contain both the build and image directives.
 - services:
 - webapp:
 - build: .
 - image: phpapp:v2
- If build: . is already present, no changes are required.

```
services:
  webapp:
    build: .
    image: phpapp:v2
    container_name: php-webapp

    ports:
      - "8080:80"

    depends_on:
      - db

    volumes:
      - php-data:/var/www/html
```

- Build and Start the Containers

```
sudo docker compose up --build -d
```

- Docker Compose will:
- Build the PHP image from the Dockerfile.
- Tag it as phpapp:v2.
- Create and start the containers.
- Verify the Running Containers

```
sudo docker ps
```

- Verify the Image

```
sudo docker images
```

```
ubuntu@ip-172-31-45-84:~/app2$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
672f088e7bb8   phpapp:v2     "docker-php-entrypoi..." 43 seconds ago Up 41 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   php-web
0319f2c85cfa   mysql:8.0     "docker-entrypoint.s..." 43 seconds ago Up 42 seconds 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp   mysql-i
nstance
ubuntu@ip-172-31-45-84:~/app2$ sudo docker images
IMAGE          ID                DISK USAGE  CONTENT SIZE  EXTRA
mysql:8.0      7dcdde01f13b     1.1GB       249MB        In Use
php:8.2-apache 408948e94ce2     714MB       183MB
phpapp:v2      cf606b3db30c     708MB       176MB        In Use
ubuntu@ip-172-31-45-84:~/app2$
```

- Stop the Containers

```
sudo docker compose down
```